

Dapr Observability with Prometheus

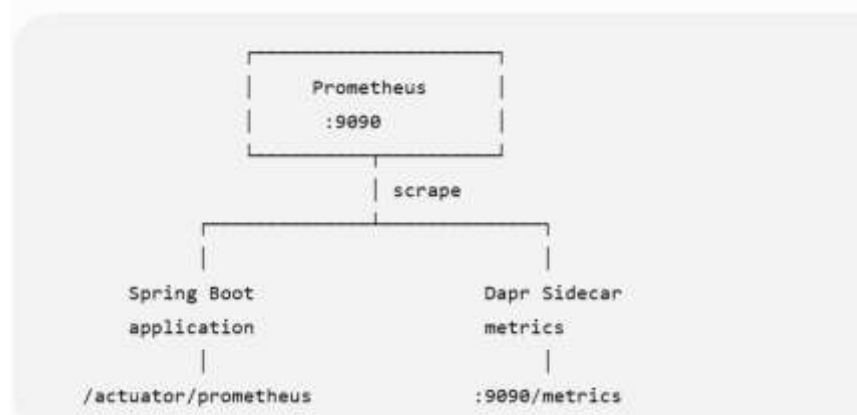
Practical Setup, PromQL Query Guide & Dashboard Reference

Table of Contents

1. Overview — Two Metrics Sources
2. Step 1 — Add the Prometheus Container
3. Step 2 — Start Prometheus
4. Step 3 — Verify Prometheus Is Running and Scraping Dapr
5. Monitoring in Three Dimensions: Traffic, Latency, Errors
6. Main HTTP Request Metric
7. Payment → Notification Traffic
8. Requests Per Second and Per Minute
9. Latency: Average and Percentile Response Time
10. Errors
11. One Important Distinction in the Architecture
12. Learning Roadmap
13. First Dashboard Layout
14. Dapr CPU Usage
15. Dapr Memory Metrics
16. Payment + Notification — Individual and Combined Queries
17. Worked Example Queries and Results
18. HTTP Status Code Analysis
19. Practical Insights
20. Latency / Histogram Metrics — General Pattern
21. Prometheus Rules
22. Useful URLs
23. Test and Operational Commands
24. Final Prometheus Flow
25. Exact Step 1 → Step 2 → Step 3 Summary

1. Overview — Two Metrics Sources

The important thing is that there are two different metrics sources we should expose:



Prometheus scrapes both the Spring Boot application and the Dapr sidecar as separate metrics sources.

Dapr itself uses port 9090 for metrics by default — we should not expose three Dapr sidecars all using the same host port.

```
Order Service
  Spring Boot :8080
  Dapr HTTP  :3502
  Dapr Metrics :9092
```

```
Payment Service
  Spring Boot :8081
  Dapr HTTP  :3501
  Dapr Metrics :9091
```

```
Notification Service
  Spring Boot :8082
  Dapr HTTP  :3500
  Dapr Metrics :9090
```

Then Prometheus can scrape:

```
Dapr Order      localhost:9092
Dapr Payment    localhost:9091
Dapr Notification localhost:9090

Order App       localhost:8080/actuator/prometheus
Payment App     localhost:8081/actuator/prometheus
Notification App localhost:8082/actuator/prometheus
```

Per-service port plan: Spring Boot, Dapr HTTP, and Dapr Metrics ports for Order, Payment and Notification services, and the resulting Prometheus scrape targets.

2. Step 1 — Add the Prometheus Container

Let's do this one step at a time. Create this directory structure in your existing project:

```
dapr-observability-poc
|
├── prometheus
│   └── prometheus.yml
|
├── dapr-order-service
├── dapr-payment-service
├── dapr-notification-service
├── dapr
└── dapr.yaml
```

Project root (as used in the practical run-through):

```
D:\Teju-AppData\SpringMicroservices-Projects\dapr-observability-poc
```

Open PowerShell in this directory:

```
cd D:\Teju-AppData\SpringMicroservices-Projects\dapr-observability-poc
```

Create prometheus/prometheus.yml

The file contains the Prometheus scrape configuration for the Dapr metrics endpoints.

```
global:
  scrape_interval: 15s

scrape_configs:

  - job_name: dapr-payment
    metrics_path: /metrics
    static_configs:
      - targets:
        - host.docker.internal:9191

  - job_name: dapr-notification
    metrics_path: /metrics
    static_configs:
      - targets:
        - host.docker.internal:9190
```

Use the actual Dapr metrics ports configured in the current POC. The important configuration is that Prometheus reaches each Dapr sidecar's /metrics endpoint.

Why host.docker.internal?

Because Prometheus runs inside Docker, while the Dapr sidecars in this setup expose their metrics on the Windows host. Therefore:

```
Prometheus container
    ↓
host.docker.internal
    ↓
Windows host
    ↓
Dapr /metrics
```

3. Step 2 — Start Prometheus

From the project root, run the following PowerShell command:

```
docker run -d `
  --name prometheus `
  -p 9095:9090 `
  -v "${PWD}\prometheus\prometheus.yml:/etc/prometheus/prometheus.yml" `
  prom/prometheus
```

Equivalent single-line form:

```
docker run -d --name prometheus -p 9095:9090 -v
"${PWD}\prometheus\prometheus.yml:/etc/prometheus/prometheus.yml" prom/prometheus
```

Or the easy way — use the Dapr multi-app configuration:

```
dapr run -f .
```

Note: dapr run -f. does not replace Prometheus itself unless dapr.yaml explicitly defines Prometheus as an app/container process.

What each Docker option means

Option	Meaning
-d	Run Prometheus in detached/background mode
--name prometheus	Give the container the name prometheus
-p 9095:9090	Map Windows host port 9095 to Prometheus container port 9090
-v ...	Mount the project's prometheus.yml into the Prometheus container
prom/prometheus	Official Prometheus Docker image

The mapping is:

Windows	Docker
localhost:9095	→ Prometheus :9090

Also note the equivalent Host → Container mapping:

Host	Container
9095	→ 9090

Therefore, Prometheus is accessed from the Windows browser using:

```
http://localhost:9095
```

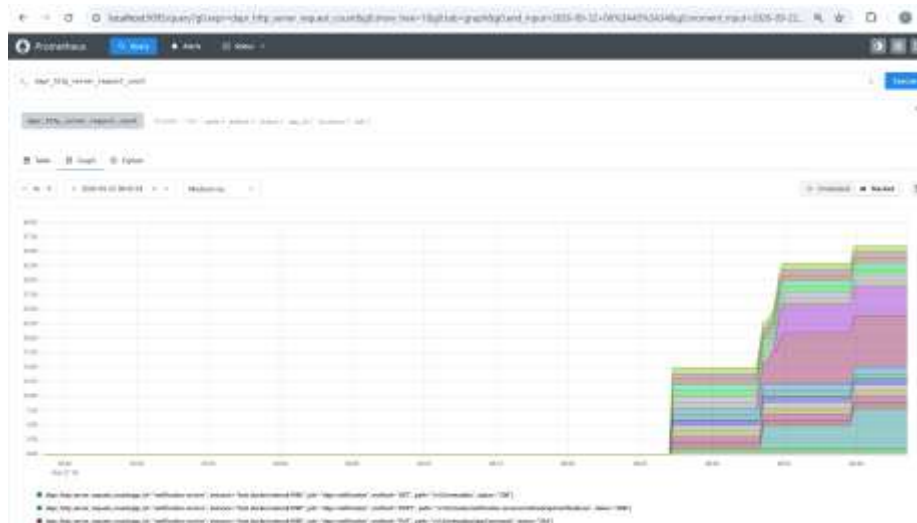
So your Prometheus UI will be reachable at the same address.

Verify the container

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}\t{{.Status}}"
```

Expected result:

NAMES	IMAGE	PORTS
prometheus	prom/prometheus	0.0.0.0:9095->9090/tcp



Prometheus web UI graph for `dapr_http_server_request_count` once traffic starts flowing — multiple stacked series for different labels.

4. Step 3 — Verify Prometheus Is Running and Scraping Dapr

4.1 Open Prometheus

Open:

```
http://localhost:9095
```

This opens the Prometheus web UI.

4.2 Check Prometheus Targets

Open:

```
http://localhost:9095/targets
```

Look for:

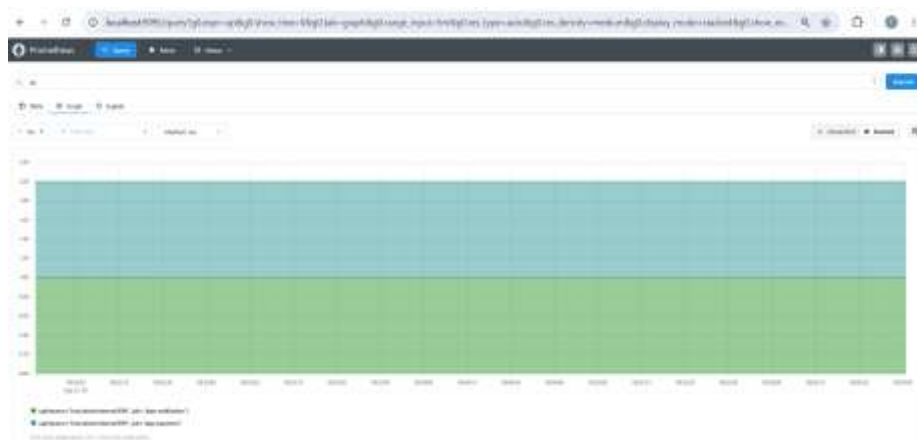
```
dapr-payment
```

```
dapr-notification
```

The target state should be UP. You can also run:

```
up
```

Meaning: up = 1 means Prometheus successfully scraped the target; up = 0 means the target could not be successfully scraped.



The up query, showing both `dapr-payment` and `dapr-notification` targets reporting as up (stacked view).

4.3 Explore Available Metrics

In the Prometheus query page, run:

```
{__name__=~".+"}
```

This allows exploration of the metric names currently stored by Prometheus. For this POC, important Dapr metrics include:

```
dapr_http_server_request_count
```

and process/runtime metrics such as:

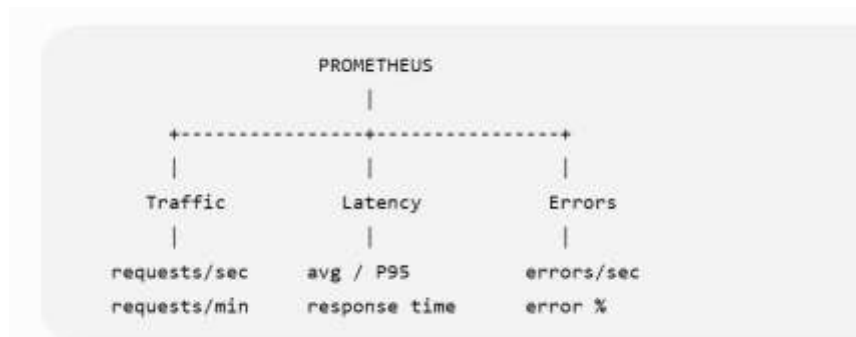
```
process_cpu_seconds_total
process_resident_memory_bytes
go_memstats_heap_alloc_bytes
go_memstats_heap_inuse_bytes
go_memstats_stack_inuse_bytes
```

Sequence for initial exploration in the Prometheus GUI:

```
1. up
   ↓
2. {__name__=~".+"}
   ↓
3. jvm_memory_used_bytes
   ↓
4. process_cpu_usage
   ↓
5. http_server_requests_seconds_count
   ↓
6. rate(http_server_requests_seconds_count[5m])
   ↓
7. histogram_quantile(...) ← latency
```

5. Monitoring in Three Dimensions: Traffic, Latency, Errors

Now that you have the raw Dapr metrics in Prometheus, the next step is to learn how to turn those metrics into operational insights. For your POC, think about it in this order:



Prometheus metrics organized into Traffic, Latency and Errors.

For the Payment → Notification flow, the recommended learning order is these six measurements first: requests/sec, requests/min, filtering out Dapr's own metadata traffic, average response time, latency units, and P95 response time.

6. Main HTTP Request Metric

The primary Dapr HTTP metric studied was:

```
dapr_http_server_request_count
```

This is a counter — it represents the cumulative number of HTTP server requests represented by each metric series. A counter continually increases:

```
100
101
102
103
...
```

Example:

```
dapr_http_server_request_count{
  app_id="payment-service"
}
```

The result can contain multiple series because the metric has labels such as:

```
app_id
method
path
status
instance
job
```

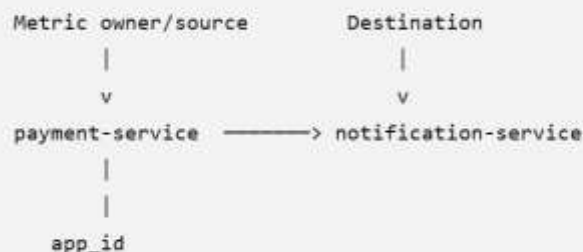
6.1 Understanding what this metric represents

This is a Dapr service invocation request. It means:



Payment Service asks its Dapr sidecar to invoke notification-service; the request travels sidecar-to-sidecar before reaching Notification Service.

Therefore, this metric is tracking the Dapr HTTP request associated with the invocation of Notification Service. It is not tracking something like POST /api/payments coming from your external client. The metric belongs to the Payment Service's Dapr sidecar. So you can think of it as:



app_id identifies the metric's owner/source (payment-service); the path label identifies the destination (notification-service).

So:

```
dapr_http_server_request_count{
  app_id="payment-service",
  path="/v1.0/invoke/notification-service/method/api/notifications",
  status="200"
}
```

means you're looking at the successful HTTP/Dapr invocation traffic for that path, rather than saying "Payment Service's own /api/payments endpoint returned 200." That's a very important distinction.

6.2 Two URLs, Two Different Requests

You potentially have two completely different requests.



External request → Payment Service (business request) vs. Payment Service → Notification Service via Dapr (service invocation).

Request A — incoming business request:

```
Client
|
| POST /api/payments
v
```

Payment Service

This answers: How many requests is Payment Service receiving from clients?

Request B — outgoing service invocation:

```
Payment Service
|
| POST /v1.0/invoke/notification-service/method/api/notifications
v
```

Notification Service

This answers: How many times is Payment Service invoking Notification Service through Dapr? The queries in this guide target Request B.

In plain English, a query such as:

```
rate(
  dapr_http_server_request_count{
    app_id="payment-service",
    path="/v1.0/invoke/notification-service/method/api/notifications"
  }[5m]
)
```

can be read as: "For the Dapr sidecar associated with payment-service, calculate the per-second rate of HTTP requests matching the Dapr invocation path that invokes notification-service, over the last 5 minutes." That's why `app_id = payment-service`, but `path = /v1.0/invoke/notification-service/...` — they represent different dimensions:

Label	Meaning
app_id="payment-service"	Which Dapr app/sidecar's metric
path="/v1.0/invoke/notification-service/..."	Which HTTP/Dapr endpoint/request
method="POST"	HTTP method
status="200"	HTTP result status
instance="host.docker.internal:9191"	Metrics endpoint/instance being scraped

7. Payment → Notification Traffic

The Dapr service invocation path studied was:

```
/v1.0/invoke/notification-service/method/api/notifications
```

Query:

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      path="/v1.0/invoke/notification-service/method/api/notifications"
    }[5m]
  )
)
```

The flow is:

```
Payment Service
  ↓
Payment Dapr Sidecar
  ↓
Dapr Service Invocation
  ↓
Notification Dapr Sidecar
  ↓
Notification Service
```

Therefore, to generate this particular traffic, trigger the Payment Service flow that invokes Notification. Calling Notification directly does not represent the Payment → Notification invocation.

Why sum(rate(...)) was used

A query such as:

```
rate(
  dapr_http_server_request_count{
    app_id="payment-service"
  }[5m]
)
```

can return multiple time series because the metric contains labels such as method, path, status, instance, job. Using sum(rate(...)) combines the matching series into one total request rate:

```
rate(...)
  ↓
one rate per matching series

sum(rate(...))
  ↓
```

```
combined rate
```

8. Requests Per Second and Per Minute

Requests per second — use `rate()` because a counter cannot be read directly as a rate. For all services:

```
sum by (app_id) (
  rate(dapr_http_server_request_count[5m])
)
```

You should get something conceptually like:

```
notification-service    0.20
payment-service         0.10
```

Meaning: notification-service → 0.20 requests/sec, payment-service → 0.10 requests/sec.

Why [5m]? `rate(metric[5m])` means: calculate the average per-second increase over the last 5 minutes. This smooths out small fluctuations.

Requests per minute — multiply the per-second rate by 60:

```
sum by (app_id) (
  rate(dapr_http_server_request_count[5m])
) * 60
```

Example: payment-service 6 requests/min, notification-service 12 requests/min. Mental conversion: requests/sec × 60 = requests/min.

Don't count internal Dapr metadata requests

This is important for the POC. The metrics contain things such as:

```
/v1.0/metadata
/v1.0/metadata/appPID
/v1.0/metadata/appCommand
/v1.0/metadata/runTemplatePath
```

Those are Dapr internal/metadata traffic, not business traffic. The important business path is:

```
/v1.0/invoke/notification-service/method/api/notifications
```

Therefore, for the Payment → Notification business flow, use:

```
sum by (app_id) (
  rate(
    dapr_http_server_request_count{
      path="/v1.0/invoke/notification-service/method/api/notifications"
    }[5m]
  )
)
```

This gives the business request rate rather than mixing in Dapr management calls.

9. Latency: Average and Percentile Response Time

9.1 Average Response Time

This is where the histogram metrics become useful. Available metrics:

```
dapr_http_server_latency_count
dapr_http_server_latency_sum
dapr_http_server_latency_bucket
```

Average latency formula:

```
average latency = increase in latency sum / increase in request count
```

In Prometheus:

```

sum(
  rate(
    dapr_http_server_latency_sum{
      app_id="notification-service",
      method="POST",
      path="/v1.0/invoke/notification-service/method/api/notifications",
      status="200"
    }[5m]
  )
)
/
sum(
  rate(
    dapr_http_server_latency_count{
      app_id="notification-service",
      method="POST",
      path="/v1.0/invoke/notification-service/method/api/notifications",
      status="200"
    }[5m]
  )
)

```

This gives the average latency for successful Notification requests.

9.2 What Unit Is Latency?

This is very important. The metric has buckets like:

1, 2, 5, 10, 13, 16, 20, 25, 30, 40, 50, 65, 80, 100, 130, 160, ... 1000, 5000, 10000

For `dapr_http_server_latency`, verify the metric's documented unit before putting ms or seconds on a Grafana dashboard. The Dapr runtime metric:

`dapr_runtime_service_invocation_res_rcv_latency_ms_bucket`

explicitly says `_ms_`, so that one is milliseconds. For that metric, for example: 50 → 50 ms, 100 → 100 ms, 500 → 500 ms.

9.3 P95 Response Time

Average latency isn't enough. Suppose you have 100 requests: 99 requests → 20 ms, 1 request → 5000 ms. Average might look acceptable while one request is extremely slow. That's why production monitoring normally also looks at P50, P95, P99.

The query `histogram_quantile(0.95, rate(dapr_http_server_latency_bucket[5]))` returned "Empty query result" because of two issues.

Issue 1 — `[5]` needs a Prometheus duration such as `[5m]`, not a bare number.

Issue 2 — `histogram_quantile` needs aggregation. For the service, use:

```

histogram_quantile(
  0.95,
  sum by (le) (
    rate(
      dapr_http_server_latency_bucket{
        app_id="notification-service",
        method="POST",
        path="/v1.0/invoke/notification-service/method/api/notifications",
        status="200"
      }[5m]
    )
  )
)

```

This is the important pattern to remember:

```
histogram
↓
rate()
↓
sum by (1e)
↓
histogram_quantile()
```

10. Errors

10.1 Error Rate Per Second

For Payment → Notification, you have status="200" and status="500", so you can directly calculate 500 responses/sec:

```
sum(
  rate(
    dapr_http_server_latency_count{
      app_id="payment-service",
      method="POST",
      path="/v1.0/invoke/notification-service/method/api/notifications",
      status="500"
    }[5m]
  )
)
```

Example: 0.05 means approximately 0.05 errors/sec, or $0.05 \times 60 = 3$ errors/min.

10.2 Error Percentage

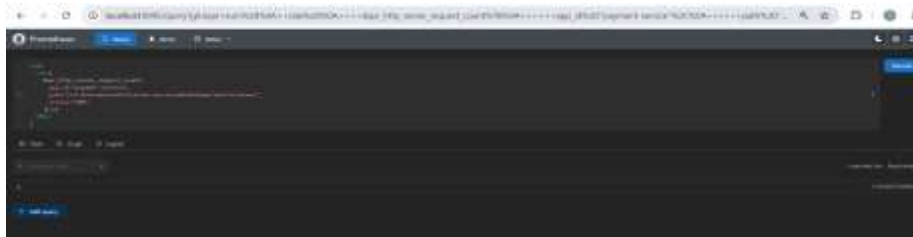
This is usually more useful than just "errors per second". Use:

```
100 *
sum(
  rate(
    dapr_http_server_latency_count{
      app_id="payment-service",
      method="POST",
      path="/v1.0/invoke/notification-service/method/api/notifications",
      status="500"
    }[5m]
  )
)
/
sum(
  rate(
    dapr_http_server_latency_count{
      app_id="payment-service",
      method="POST",
      path="/v1.0/invoke/notification-service/method/api/notifications"
    }[5m]
  )
)
```

Suppose total requests = 100/min and 500 errors = 5/min. Then error rate = $5 / 100 \times 100 = 5\%$. So a dashboard could say:

Payment → Notification	Value
Traffic	100 req/min
Avg latency	35 ms
P95 latency	80 ms
Errors	5 req/min
Error rate	5%

That's already a meaningful operational view.



Errors/min for the Payment → Notification path: `sum(rate(dapr_http_server_request_count{status="500"}[1m])) * 60.`

11. One Important Distinction in the Architecture

The flow is:

DAPR

Payment Service

|
| 1. Payment calls Dapr
v

Payment Dapr Sidecar

|
| 2. Dapr invokes Notification
v

Notification Dapr

|
| 3. Dapr calls application
v

Notification Service

Therefore there are actually different latency measurements:

- Payment-side perspective — Payment → Dapr → Notification, measured using the Payment-side metrics.
- Notification-side perspective — Notification Dapr → Notification application, measured using `dapr_http_server_latency`.
- Dapr service-invocation perspective — `dapr_runtime_service_invocation_res_recv_latency_ms_bucket`, which represents the Dapr service invocation response latency.

12. Learning Roadmap

Don't try to understand all 100+ Dapr metrics at once. Learn these in this order:

Insight	Prometheus concept
Is service alive?	up

Insight	Prometheus concept
Requests/sec	<code>rate(counter[5m])</code>
Requests/min	<code>rate(counter[5m]) * 60</code>
Average latency	<code>rate(sum) / rate(count)</code>
P50	<code>histogram_quantile(0.50, ...)</code>
P95	<code>histogram_quantile(0.95, ...)</code>
P99	<code>histogram_quantile(0.99, ...)</code>
Errors/sec	<code>rate(500_count[5m])</code>
Errors/min	<code>rate(500_count[5m]) * 60</code>
Error %	<code>5xx rate / total rate * 100</code>
Throughput	<code>requests/sec</code>
Saturation	<code>CPU/memory/Goroutines</code>
Availability	<code>up</code>

The 5 PromQL Patterns to Remember

① Counter → rate

```
rate(metric[5m])
```

Meaning: How fast is this counter increasing?

② Per minute

```
rate(metric[5m]) * 60
```

Meaning: How many events are occurring per minute?

③ Average latency

```
rate(latency_sum[5m])
/
rate(latency_count[5m])
```

Meaning: On average, how long does each request take?

④ Percentile latency

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(latency_bucket[5m])
  )
)
```

Meaning: What latency does 95% of traffic complete within?

⑤ Error percentage

```
100 *
rate(error_count[5m])
/
rate(total_count[5m])
```

Meaning: What percentage of requests are failing?

Quick Reference: Rate Conversions

Metric	Expression
Requests/sec	<code>rate(counter[5m])</code>
Requests/min	<code>rate(counter[5m]) * 60</code>
Seconds/request	<code>1 / requests_per_sec</code>
Minutes/request	<code>1 / requests_per_min</code>

13. First Dashboard Layout

For the POC, the first dashboard should eventually look like this:

DAPR OBSERVABILITY	
Payment Requests/sec 1.25	Notification Requests/sec 1.18
Payment Req/min 75	Notification Req/min 71
Avg Latency XX ms	P95 Latency XX ms
Errors/min X	Error % X%
Payment → Notification Traffic Payment → Notification Latency Payment → Notification Errors	

The key idea: don't learn Prometheus as a list of metric names. Learn it as questions about your system: How much traffic? → How fast? → How many failures? → How healthy? → Where is the latency? Once these questions can be answered from the Dapr metrics, the next steps are Grafana dashboards, alerts, and Zipkin distributed tracing.

14. Dapr CPU Usage

The process CPU counter studied was:

```
process_cpu_seconds_total{
  job="dapr-payment"
}
```

Convert the CPU counter into a rate:

```
rate(
  process_cpu_seconds_total{
    job="dapr-payment"
  }[5m]
)
```

The result represents CPU consumption in terms of CPU cores. For example, 0.03 approximately means 0.03 CPU cores. It should not automatically be interpreted as "3% of the entire host."

15. Dapr Memory Metrics

15.1 Resident Memory

The process resident memory metric studied was:

```
process_resident_memory_bytes{  
  job="dapr-payment"  
}
```

Convert bytes to MB:

```
process_resident_memory_bytes{  
  job="dapr-payment"  
} / 1024 / 1024
```

Meaning: physical resident memory currently occupied by the Dapr process — useful for monitoring the overall physical memory footprint of the sidecar.

15.2 Go Heap Memory

Because Dapr is implemented in Go, Go runtime memory metrics are also exposed.

Heap allocated:

```
go_memstats_heap_alloc_bytes{  
  job="dapr-payment"  
} / 1024 / 1024
```

Meaning: live Go heap memory currently allocated.

Heap in use:

```
go_memstats_heap_inuse_bytes{  
  job="dapr-payment"  
} / 1024 / 1024
```

Meaning: Go heap spans currently in use.

Go stack memory:

```
go_memstats_stack_inuse_bytes{  
  job="dapr-payment"  
} / 1024 / 1024
```

Meaning: memory currently used by Go stacks.

Important distinction:

```
process_resident_memory_bytes  
  ↓  
overall physical resident memory of Dapr  
  
go_memstats_heap_alloc_bytes  
  ↓  
live Go heap allocations  
  
go_memstats_heap_inuse_bytes  
  ↓  
Go heap spans in use  
  
go_memstats_stack_inuse_bytes  
  ↓  
Go stack memory in use
```

15.3 Full Memory Metric Reference

Concise summary of the main memory-related metrics and PromQL queries for Dapr sidecar observability:

Metric / Purpose	PromQL Query	Unit	What it tells you
Process resident memory ★	<code>process_resident_memory_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Physical memory currently occupied by the Dapr sidecar
Process virtual memory	<code>process_virtual_memory_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Virtual memory address space used by the sidecar
Go heap allocated	<code>go_memstats_heap_alloc_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Heap memory currently allocated to live Go objects
Go heap in use	<code>go_memstats_heap_inuse_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Memory in spans currently being used for Go heap objects
Go heap system memory	<code>go_memstats_heap_sys_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Memory obtained from the OS for the Go heap
Total allocated over lifetime	<code>go_memstats_alloc_bytes_total{job="dapr-payment"} / 1024 / 1024</code>	MB	Cumulative memory allocated by Go since process start; not current memory
Heap objects	<code>go_memstats_heap_objects{job="dapr-payment"}</code>	Count	Number of allocated objects currently on the Go heap
Stack memory in use	<code>go_memstats_stack_inuse_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Memory currently used for Go goroutine stacks
Stack memory obtained	<code>go_memstats_stack_sys_bytes{job="dapr-payment"} / 1024 / 1024</code>	MB	Memory obtained from OS for goroutine stacks
GC CPU percentage	<code>rate(go_memstats_gc_cpu_fraction{job="dapr-payment"}[5m]) * 100</code>	Fraction	CPU time spent in garbage collection

15.4 Most Important for the Dashboard

If the goal is simply "How much memory is my Dapr sidecar using?", use:

```
process_resident_memory_bytes{job="dapr-payment"} / 1024 / 1024
```

For notification:

```
process_resident_memory_bytes{job="dapr-notification"} / 1024 / 1024
```

15.5 Memory Dashboard — 4 Grafana Panels

Dashboard Panel	Query
Sidecar Memory (MB)	<code>process_resident_memory_bytes{job="dapr-payment"} / 1024 / 1024</code>
Go Heap Used (MB)	<code>go_memstats_heap_alloc_bytes{job="dapr-payment"} / 1024 / 1024</code>
Go Heap In Use (MB)	<code>go_memstats_heap_inuse_bytes{job="dapr-payment"} / 1024 / 1024</code>
Goroutine Stack Memory (MB)	<code>go_memstats_stack_inuse_bytes{job="dapr-payment"} / 1024 / 1024</code>



Grafana dashboard with the four memory panels: Dapr Sidecar Memory, Dapr Go Heap Allocated, Dapr Goroutine Stack Memory, and Payment vs Notification memory use.

16. Payment + Notification — Individual and Combined Queries

For the two Dapr sidecars — payment and notification — you can have both individual and combined memory/CPU queries.

Metric	Payment	Notification	Combined / Total
Resident Memory (MB)	<code>process_resident_memory_bytes{job="dapr-payment"} / 1024 / 1024</code>	<code>process_resident_memory_bytes{job="dapr-notification"} / 1024 / 1024</code>	<code>sum(process_resident_memory_bytes{job=~"dapr-payment dapr-notification"}) / 1024 / 1024</code>
Go Heap Allocated (MB)	<code>go_memstats_heap_alloc_bytes{job="dapr-payment"} / 1024 / 1024</code>	<code>go_memstats_heap_alloc_bytes{job="dapr-notification"} / 1024 / 1024</code>	<code>sum(go_memstats_heap_alloc_bytes{job=~"dapr-payment dapr-notification"}) / 1024 / 1024</code>
Goroutines	<code>go_goroutines{job="dapr-payment"}</code>	<code>go_goroutines{job="dapr-notification"}</code>	<code>sum(go_goroutines{job=~"dapr-payment dapr-notification"})</code>
CPU cores used	<code>rate(process_cpu_seconds_total{job="dapr-payment"}[5m])</code>	<code>rate(process_cpu_seconds_total{job="dapr-notification"}[5m])</code>	<code>sum(rate(process_cpu_seconds_total{job=~"dapr-payment dapr-notification"}[5m]))</code>

Metric	Payment	Notification	Combined / Total
CPU % of one core	<code>rate(process_cpu_seconds_total{job="dapr-payment"}[5m]) * 100</code>	<code>rate(process_cpu_seconds_total{job="dapr-notification"}[5m]) * 100</code>	<code>sum(rate(process_cpu_seconds_total{job=~"dapr-payment dapr-notification"}[5m])) * 100</code>

★ Combined memory query

If you want: "How many MB are both Dapr sidecars consuming together?", use:

```
sum(
  process_resident_memory_bytes{
    job=~"dapr-payment|dapr-notification"
  }
) / 1024 / 1024
```

Example:

Payment sidecar	85 MB
Notification sidecar	78 MB

Combined	163 MB

★ Combined CPU usage

If you want: "How many CPU cores are both sidecars consuming together?"

```
sum(
  rate(
    process_cpu_seconds_total{
      job=~"dapr-payment|dapr-notification"
    }[5m]
  )
)
```

Example:

Payment	0.08 core
Notification	0.05 core

Total	0.13 core

If you want that expressed as a percentage of one CPU core:

```
sum(
  rate(
    process_cpu_seconds_total{
      job=~"dapr-payment|dapr-notification"
    }[5m]
  )
) * 100
```

That would show: $0.13 \times 100 = 13\%$

⚠ CPU % of the entire machine

Suppose the Windows machine has 16 logical processors. Then the combined sidecar CPU percentage of the whole machine would be:

```
(
  sum(
    rate(
```

```

    process_cpu_seconds_total{
      job=~"dapr-payment|dapr-notification"
    }[5m]
  )
)
/ 16
) * 100

```

So if both sidecars consume 0.13 cores: $0.13 / 16 \times 100 = 0.8125\%$. That means the two Dapr sidecars together are consuming approximately 0.81% of the machine's total logical CPU capacity.

17. Worked Example Queries and Results

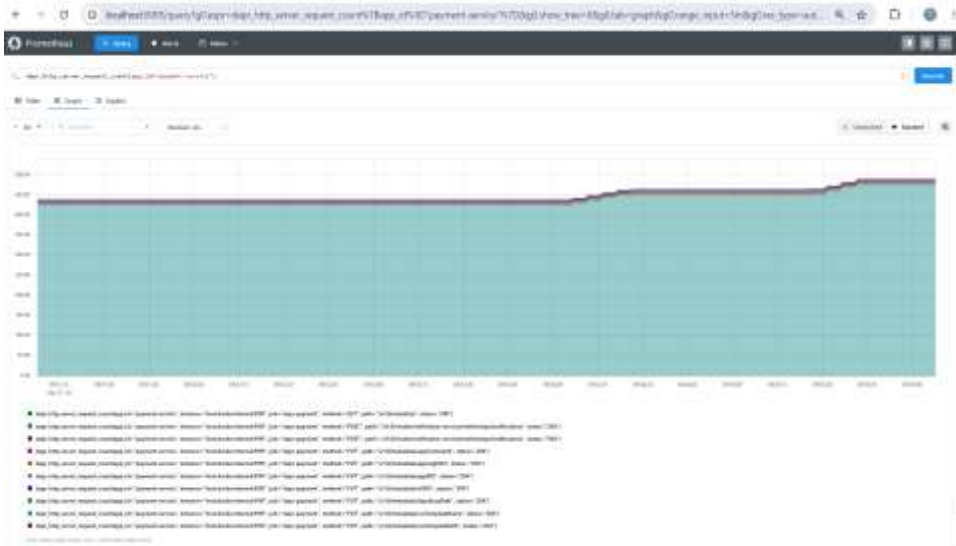
These are practical examples of the metrics traced during the POC, matched to their Prometheus screenshots.

1. How many total HTTP server requests across all matching time series has the payment-service Dapr sidecar accumulated?

```
sum(dapr_http_server_request_count{app_id="payment-service"})
```

Same response in detail — bifurcated by status (shows all 200, 500, etc.):

```
dapr_http_server_request_count{app_id="payment-service"}
```



Raw dapr_http_server_request_count series for app_id="payment-service", broken down by method, path and status.

2. Payment → Notification successful invocation rate (status 200) per second

How many successful requests per second have been occurring, based on the last 5 minutes?

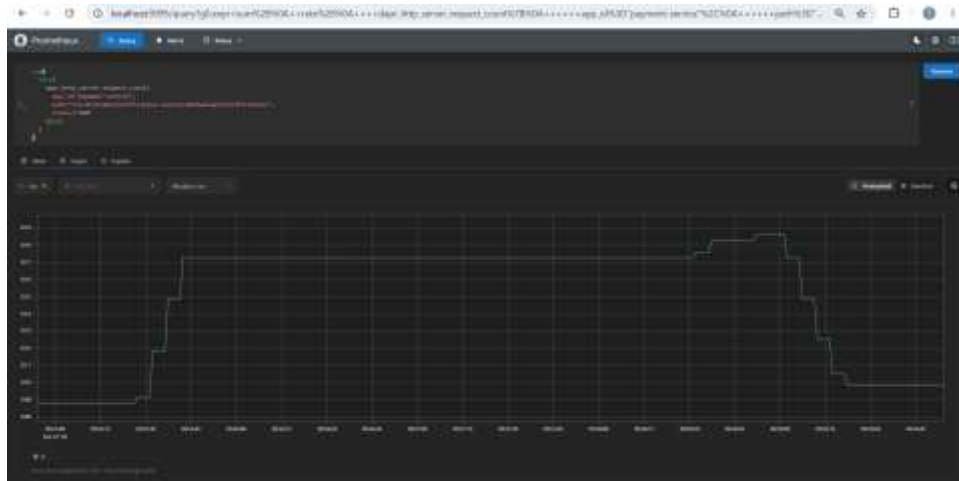
```

sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      path="/v1.0/invoke/notification-service/method/api/notifications",
      status="200"
    }[5m]
  )
)

```



Raw counter value for the filtered Payment → Notification path with status="200" (table view).



$sum(rate(...status="200"...[5m]))$ plotted over time for the Payment → Notification path.

3. Payment → Notification total invocation rate/sec

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      path="/v1.0/invoke/notification-service/method/api/notifications"
    }[5m]
  )
)
```

4. Payment → Notification calls per minute

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      path="/v1.0/invoke/notification-service/method/api/notifications"
    }[5m]
  )
) * 60
```

5. Error rate

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      path="/v1.0/invoke/notification-service/method/api/notifications",
      status=~"5.."
    }[5m]
  )
)
```

Raw current counter value, matching these labels (a cumulative count) — requests per second have been occurring, based on the last 5 minutes:

```
dapr_http_server_request_count{
  app_id="payment-service",
  path="/v1.0/invoke/notification-service/method/api/notifications",
  status="200"
}
```

And the 500 equivalent:

```
dapr_http_server_request_count{
  app_id="payment-service",
  path="/v1.0/invoke/notification-service/method/api/notifications",
  status="500"
}
```

or, aggregated:

```
sum(dapr_http_server_request_count{
  app_id="payment-service",
  path="/v1.0/invoke/notification-service/method/api/notifications",
  status="500"
})
```

Note: `dapr_http_server_request_count{status="200"}` alone returns raw values that are not specific to one API — the `path` label must be added to filter down to the API of interest.

18. HTTP Status Code Analysis

Because status is a metric label, traffic can be separated by status class.

Successful 2xx requests:

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      status=~"2.."
    }[5m]
  )
)
```

4xx requests:

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      status=~"4.."
    }[5m]
  )
)
```

5xx requests:

```
sum(
  rate(
    dapr_http_server_request_count{
      app_id="payment-service",
      status=~"5.."
    }[5m]
  )
)
```

Meaning: 2xx → successful responses, 4xx → client/request-side errors, 5xx → server-side errors.

19. Practical Insights

Insight 1 — Prometheus does not create the Dapr metrics

```
Dapr
↓
/metrics
```

Prometheus scrapes and stores them:

```
Dapr
↓
Prometheus
```

Grafana later visualizes them:

```
Dapr
↓
Prometheus
↓
Grafana
```

Insight 2 — Counters are normally converted into rates for traffic monitoring

Raw: `dapr_http_server_request_count` (cumulative counter)

For traffic: `rate(dapr_http_server_request_count[5m])` (more useful)

Insight 3 — Labels produce multiple metric series

`app_id, method, path, status, job, instance`

Therefore `dapr_http_server_request_count{app_id="payment-service"}` can return multiple rows/series.

Insight 4 — `sum()` aggregates those series

`sum(rate(...))` → produces a combined rate across the matching series

Insight 5 — Payment → Notification traffic is generated from Payment

```
Payment
↓
Payment Dapr
↓
Notification Dapr
↓
Notification
```

So the Payment API/request that triggers the notification must be called to generate that Dapr invocation metric.

Insight 6 — CPU and memory are different metric types

CPU: `rate(process_cpu_seconds_total[5m])` – derived from a counter, represents CPU consumption rate

Memory: `process_resident_memory_bytes` – a gauge representing current resident memory

20. Latency / Histogram Metrics — General Pattern

Dapr also exposes latency-related histogram metrics. The general Prometheus histogram percentile pattern is:

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(<latency_metric_bucket>[5m])
  )
)
```

Meaning: P95 latency approximately represents the latency below which 95% of observed requests completed during the selected time window. The exact latency metric name should be selected from the metrics exposed by the Dapr version running in the current POC.

21. Prometheus Rules

Prometheus rules can be viewed at:

```
http://localhost:9095/rules?health=ok
```

This is useful for checking configured recording and alerting rules.

22. Useful URLs

Purpose	URL
Prometheus UI	http://localhost:9095
Targets	http://localhost:9095/targets
Rules	http://localhost:9095/rules?health=ok
Zipkin	http://localhost:9411
Grafana (after starting)	http://localhost:3000

23. Test and Operational Commands

Traffic-generation commands used during the POC:

```
Invoke-RestMethod -Method Post -Uri "http://localhost:8081/api/payments"
```

```
Invoke-RestMethod -Method Post -Uri "http://localhost:3500/v1.0/invoke/notification-  
service/method/api/notifications"
```

```
Measure-Command {Invoke-RestMethod -Method Post -Uri  
"http://localhost:3501/v1.0/invoke/notification-service/method/api/notifications?delay=10000"}
```

```
Invoke-RestMethod -Method Post -Uri "http://localhost:3501/v1.0/invoke/notification-  
service/method/api/notifications?fail=true"
```

```
Invoke-RestMethod -Method Post -Uri "http://localhost:3501/v1.0/invoke/notification-  
service/method/api/notifications?transientFailure=true"
```

Dapr multi-app run and stop:

```
dapr run -f .  
dapr run -f .\dapr.yaml
```

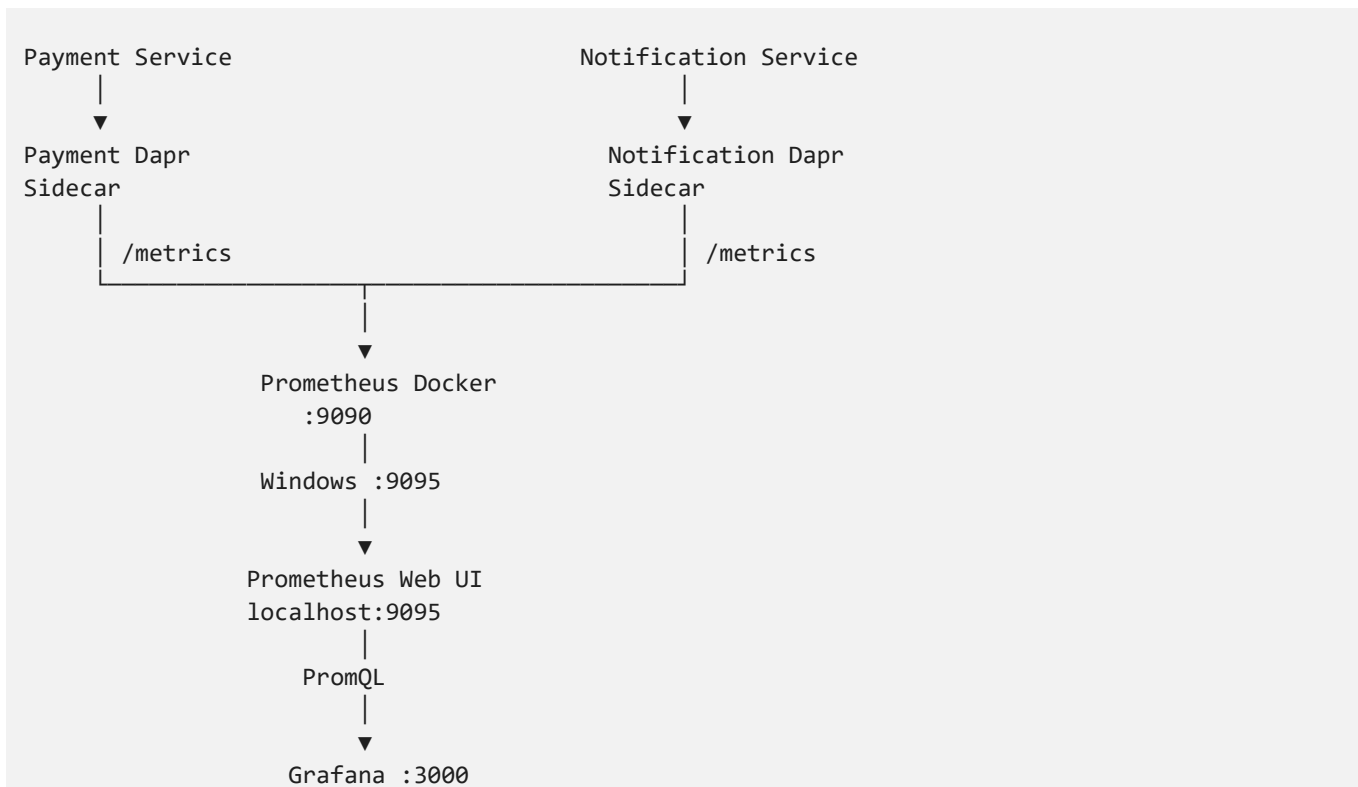
```
dapr stop --app-id notification-service  
dapr stop --app-id payment-service
```

```
Get-Process -Id 7196
```

Restart Prometheus:

```
docker restart prometheus
```

24. Final Prometheus Flow



25. Exact Step 1 → Step 2 → Step 3 Summary

Step 1 — Configure Prometheus

File:

```
prometheus\prometheus.yml
```

Configure the Dapr /metrics targets.

Step 2 — Start Prometheus

From:

```
D:\Teju-AppData\SpringMicroservices-Projects\dapr-observability-poc
```

Run:

```
docker run -d --name prometheus -p 9095:9090 -v
"${PWD}\prometheus\prometheus.yml:/etc/prometheus/prometheus.yml" prom/prometheus
```

Verify:

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}\t{{.Status}}"
```

Step 3 — Verify and query Prometheus

Open:

```
http://localhost:9095
```

Check:

```
http://localhost:9095/targets
```

Then run PromQL such as:

```
dapr_http_server_request_count
```

and:

```
sum(
  rate(
    dapr_http_server_request_count{
```

```
        app_id="payment-service"  
    }[5m]  
)  
)
```

This completes the practical Prometheus setup before moving to Grafana.